ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С.П.КОРОЛЕВА»

Институт «Информатики и кибернетики»

Специальность «Фотоника и оптоинформатика»

Отчет по лабораторной работе № 3

Выполнила: Гамаюнова Д.И.,
группа 6201-120303D
Проверил: преподаватель Борисов Д.С.

Самара 2025

Задание 1

Exception – проверяемый класс исключений, который используется для создания своих проверяемых исключений.

IndexOutOfBoundsException – используется при недопустимом индексе для коллекции, списка и т.д.

ArrayIndexOutOfBoundsException – используется при недопустимом индексе для массива.

IllegalArgumentException – используется, когда методу передан некорректный аргумент.

IllegalStateException – используется, когда метод вызван в некорректном состоянии объекта.

Задание 2

Создаем два класса исключений:

- FunctionPointIndexOutOfBoundsException наследуется от IndexOutOfBoundsException (непроверяемое исключение)

```
1   package functions;
2
3   public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {  no usages
4       public FunctionPointIndexOutOfBoundsException() {  no usages
5           super();
6       }
7
8       public FunctionPointIndexOutOfBoundsException(String s) {  no usages
9           super(s);
10      }
11
12      public FunctionPointIndexOutOfBoundsException(int index) {  no usages
13          super("Function point index out of range: " + index);
14      }
15  }
```

- InappropriateFunctionPointException наследуется от Exception (проверяемое исключение)

```java
package functions;

public class InappropriateFunctionPointException extends Exception {  no usages
    public InappropriateFunctionPointException() {  no usages
        super();
    }

    public InappropriateFunctionPointException(String message) {  no usages
        super(message);
    }

    public InappropriateFunctionPointException(String message, Throwable cause) {  no usages
        super(message, cause);
    }

    public InappropriateFunctionPointException(Throwable cause) {  no usages
        super(cause);
    }
}
```

Задание 3

Конструкторы теперь выбрасывают IllegalArgumentException если левая граница ≥ правой границы, количество точек < 2

```java
public TabulatedFunction(double leftX, double rightX, int pointsCount) {  1 usage
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница области определения >= правой");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек меньше двух");
    }
    this.points = new FunctionPoint[pointsCount];
    this.size = pointsCount;
    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        points[i] = new FunctionPoint(x,  y: 0.0);
    }
}
public TabulatedFunction(double leftX, double rightX, double[] values) {  1 usage
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница области определения больше или равна правой");
    }
    if (values.length < 2) {
        throw new IllegalArgumentException("Количество точек меньше двух");
    }
    int n = values.length;
    this.points = new FunctionPoint[n];
    this.size = n;
    double step = (rightX - leftX) / (n - 1);
    for (int i = 0; i < n; i++) {
        double x = leftX + i * step;
        points[i] = new FunctionPoint(x, values[i]);
    }
}
```

Измененные *методы*:

getPoint

```
public FunctionPoint getPoint(int index) {  no usages
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс точки выходит за границы: " + index);
    }
    return new FunctionPoint(points[index]);
}
```

setPoint

```
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {  1 usage
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс точки выходит за границы: " + index);
    }
    if ((index > 0 && point.getX() <= points[index - 1].getX()) || (index < size - 1 && point.getX() >= points[index + 1].getX())) {
        throw new InappropriateFunctionPointException("Координата X точки нарушает упорядоченность точек функции");
    }
    points[index] = new FunctionPoint(point);
}
```

getPointX

```
public double getPointX(int index) {  6 usages
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс точки выходит за границы: " + index);
    }
    return points[index].getX();
}
```

setPointX

```
public void setPointX(int index, double x) throws InappropriateFunctionPointException {  no usages
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс точки выходит за границы: " + index);
    }
    if ((index > 0 && x <= points[index - 1].getX()) ||
            (index < size - 1 && x >= points[index + 1].getX())) {
        throw new InappropriateFunctionPointException("Координата X точки нарушает упорядоченность точек функции");
    }

    points[index].setX(x);
}
```

getPointY

```
public double getPointY(int index) {  5 usages
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс точки выходит за границы: " + index);
    }
    return points[index].getY();
}
```

setPointY

```
public void setPointY(int index, double y) {  1 usage
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс точки выходит за границы: " + index);
    }
    points[index].setY(y);
}
```

deletePoint

```
public void deletePoint(int index) {  1 usage
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс точки выходит за границы: " + index);
    }
    if (size <= 2) {
        throw new IllegalStateException("Невозможно удалить точку: должно остаться минимум 2 точки");
    }

    for (int i = index; i < size - 1; i++) {
        points[i] = points[i + 1];
    }
    points[size - 1] = null;
    size--;
}
```

addPoint

```
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {  3 usages
    for (int i = 0; i < size; i++) {
        if (Math.abs(points[i].getX() - point.getX()) < 1e-10) {
            throw new InappropriateFunctionPointException("Точка с координатой X=" + point.getX() + " уже существует");
        }
    }

    if (size == points.length) {
        FunctionPoint[] newPoints = new FunctionPoint[size * 2];
        System.arraycopy(points,  srcPos: 0, newPoints,  destPos: 0, size);
        points = newPoints;
    }

    int insertIndex = 0;
    double x = point.getX();
    while (insertIndex < size && points[insertIndex].getX() < x) {
        insertIndex++;
    }

    for (int i = size; i > insertIndex; i--) {
        points[i] = points[i - 1];
    }
    points[insertIndex] = new FunctionPoint(point);
    size++;
}
```

Задание 4

1. Описание класса элементов списка FunctionNode

```
private static class FunctionNode {
    FunctionPoint point;
    FunctionNode prev;
    FunctionNode next;
    public FunctionNode() {
        this.point = null;
        this.prev = this;
        this.next = this;
    }
    public FunctionNode(FunctionPoint point) {
        this.point = point;
        this.prev = this;
        this.next = this;
    }
    public FunctionNode(FunctionPoint point, FunctionNode prev, FunctionNode next) {
        this.point = point;
        this.prev = prev;
        this.next = next;
    }
}
```

Я поместила класс FunctionNode в класс LinkedListTabulatedFunction, так как:

- FunctionNode - внутренняя деталь реализации списка
- Без списка узел не имеет смысла и не должен существовать отдельно

Класс FunctionNode я сделала приватным, чтобы только LinkedListTabulatedFunction мог использовать узлы (полная инкапсуляция) и статичным, чтобы узлы не хранили ссылку на внешний объект (экономия памяти).

Обоснование инкапсуляции в FunctionNode:

- Поля без геттеров/сеттеров - так как используется только внутри родительского класса
- Прямой доступ к полям - проще, быстрее, читаемее

2. Методы для работы с объектами класса LinkedListTabulatedFunction

```
private FunctionNode head;  12 usages
private int size;  23 usages
private FunctionNode lastAccessedNode;  3 usages
private int lastAccessedIndex;  8 usages
```

3. private FunctionNode getNodeByIndex(int index) возвращает ссылку на объект элемента списка по его номеру.

```
private FunctionNode getNodeByIndex(int index) {  9 usages
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс за пределами списка: " + index);
    }
    FunctionNode currentNode;
    int currentIndex;
    if (lastAccessedIndex != -1) {
        int distanceFromLast = Math.abs(index - lastAccessedIndex);
        int distanceFromStart = index;
        int distanceFromEnd = size - 1 - index;
        if (distanceFromLast <= distanceFromStart && distanceFromLast <= distanceFromEnd) {
            currentNode = lastAccessedNode;
            currentIndex = lastAccessedIndex;
        } else if (distanceFromStart <= distanceFromEnd) {
            currentNode = head.next;
            currentIndex = 0;
        } else {
            currentNode = head.prev;
            currentIndex = size - 1;
        }
    } else {
        currentNode = head.next;
        currentIndex = 0;
    }
    while (currentIndex != index) {
        if (currentIndex < index) {
            currentNode = currentNode.next;
            currentIndex++;
        } else {
            currentNode = currentNode.prev;
            currentIndex--;
        }
    }
    lastAccessedNode = currentNode;
    lastAccessedIndex = index;
    return currentNode;
}
```

4. private FunctionNode addNodeToTail() добавляет новый элемент в конец списка и возвращает ссылку на объект этого элемента.

```
private FunctionNode addNodeToTail() {  3 usages
    FunctionNode newNode = new FunctionNode();
    FunctionNode tail = head.prev;
    newNode.prev = tail;
    newNode.next = head;
    tail.next = newNode;
    head.prev = newNode;
    size++;
    lastAccessedIndex = -1;
    return newNode;
}
```

5. private FunctionNode addNodeByIndex(int index) добавляет новый элемент в указанную позицию списка и возвращает ссылку на объект этого элемента.

```
private FunctionNode addNodeByIndex(int index) {  1 usage
    if (index < 0 || index > size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс за пределами списка: " + index);
    }

    if (index == size) {
        return addNodeToTail();
    }

    FunctionNode nextNode = getNodeByIndex(index);
    FunctionNode prevNode = nextNode.prev;
    FunctionNode newNode = new FunctionNode();
    newNode.prev = prevNode;
    newNode.next = nextNode;
    prevNode.next = newNode;
    nextNode.prev = newNode;
    size++;
    lastAccessedIndex = -1;
    return newNode;
}
```

6. private FunctionNode deleteNodeByIndex(int index) удаляет элемент списка по номеру и возвращает ссылку на объект удаленного элемента.

```
private FunctionNode deleteNodeByIndex(int index) {  1 usage
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс за пределами списка: " + index);
    }
    if (size <= 2) {
        throw new IllegalStateException("Невозможно удалить точку: должно остаться минимум 2 точки");
    }

    FunctionNode nodeToDelete = getNodeByIndex(index);
    FunctionNode prevNode = nodeToDelete.prev;
    FunctionNode nextNode = nodeToDelete.next;
    prevNode.next = nextNode;
    nextNode.prev = prevNode;
    nodeToDelete.prev = null;
    nodeToDelete.next = null;

    size--;
    lastAccessedIndex = -1;

    return nodeToDelete;
}
```

Задание 5

Для обеспечения второй функции класса LinkedListTabulatedFunction я реализовала в классе конструкторы и методы, аналогичные конструкторам и методам класса TabulatedFunction.

```
public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) {  no usages
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница области определения больше или равна правой");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек меньше двух");
    }

    initializeList();
    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        FunctionNode newNode = addNodeToTail();
        newNode.point = new FunctionPoint(x,  y: 0.0);
    }
}

public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) {  no usages
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница области определения больше или равна правой");
    }
    if (values.length < 2) {
        throw new IllegalArgumentException("Количество точек меньше двух");
    }

    initializeList();
    double step = (rightX - leftX) / (values.length - 1);
    for (int i = 0; i < values.length; i++) {
        double x = leftX + i * step;
        FunctionNode newNode = addNodeToTail();
        newNode.point = new FunctionPoint(x, values[i]);
    }
}
```

```
public double getLeftDomainBorder() {  1 usage
    if (size == 0) return Double.NaN;
    return head.next.point.getX();
}
public double getRightDomainBorder() {  1 usage
    if (size == 0) return Double.NaN;
    return head.prev.point.getX();
}
```

```java
public double getFunctionValue(double x) {  no usages
    if (size == 0 || x < getLeftDomainBorder() - 1e-10 || x > getRightDomainBorder() + 1e-10) {
        return Double.NaN;
    }
    for (int i = 0; i < size - 1; i++) {
        FunctionNode node = getNodeByIndex(i);
        FunctionNode nextNode = node.next;
        double x0 = node.point.getX();
        double x1 = nextNode.point.getX();
        double y0 = node.point.getY();
        double y1 = nextNode.point.getY();
        if (Math.abs(x - x0) < 1e-10) return y0;
        if (Math.abs(x - x1) < 1e-10) return y1;
        if (x > x0 - 1e-10 && x < x1 + 1e-10) {
            return y0 + (y1 - y0) * (x - x0) / (x1 - x0);
        }
    }
    return Double.NaN;
}
public int getPointsCount() {  no usages
    return size;
}
public FunctionPoint getPoint(int index) {  no usages
    return new FunctionPoint(getNodeByIndex(index).point);
}
```

```java
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {  no usages
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс точки выходит за границы: " + index);
    }

    FunctionNode node = getNodeByIndex(index);

    if (index > 0) {
        FunctionNode prevNode = node.prev;
        if (point.getX() <= prevNode.point.getX()) {
            throw new InappropriateFunctionPointException("Координата X точки нарушает упорядоченность точек функции");
        }
    }
    if (index < size - 1) {
        FunctionNode nextNode = node.next;
        if (point.getX() >= nextNode.point.getX()) {
            throw new InappropriateFunctionPointException("Координата X точки нарушает упорядоченность точек функции");
        }
    }

    node.point = new FunctionPoint(point);
}

public double getPointX(int index) {  2 usages
    return getNodeByIndex(index).point.getX();
}
```

```java
public void setPointX(int index, double x) throws InappropriateFunctionPointException {  no usages
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс точки выходит за границы: " + index);
    }

    FunctionNode node = getNodeByIndex(index);
    FunctionPoint currentPoint = node.point;

    if (index > 0) {
        FunctionNode prevNode = node.prev;
        if (x <= prevNode.point.getX()) {
            throw new InappropriateFunctionPointException("Координата X точки нарушает упорядоченность точек функции");
        }
    }
    if (index < size - 1) {
        FunctionNode nextNode = node.next;
        if (x >= nextNode.point.getX()) {
            throw new InappropriateFunctionPointException("Координата X точки нарушает упорядоченность точек функции");
        }
    }

    node.point = new FunctionPoint(x, currentPoint.getY());
}

public double getPointY(int index) {  no usages
    return getNodeByIndex(index).point.getY();
}

public void setPointY(int index, double y) {  no usages
    FunctionNode node = getNodeByIndex(index);
    FunctionPoint currentPoint = node.point;
    node.point = new FunctionPoint(currentPoint.getX(), y);
}

public void deletePoint(int index) {  no usages
    deleteNodeByIndex(index);
}
```

```java
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {  no usages
    for (int i = 0; i < size; i++) {
        if (Math.abs(getPointX(i) - point.getX()) < 1e-10) {
            throw new InappropriateFunctionPointException("Точка с координатой X=" + point.getX() + " уже существует");
        }
    }

    int insertIndex = 0;
    double x = point.getX();
    while (insertIndex < size && getPointX(insertIndex) < x) {
        insertIndex++;
    }

    FunctionNode newNode = addNodeByIndex(insertIndex);
    newNode.point = new FunctionPoint(point);
}
```

Задание 6

Класс TabulatedFunction я переименовала в класс ArrayTabulatedFunction. Создала интерфейс TabulatedFunction, содержащий объявления общих методов классов ArrayTabulatedFunction и LinkedListTabulatedFunction.

TabulatedFunction.java | FunctionPointIndexOutOfBoundsException.java | InappropriateFunctionPointException.java

```java
package functions;

public interface TabulatedFunction {  no usages

    double getLeftDomainBorder();  no usages
    double getRightDomainBorder();  no usages
    double getFunctionValue(double x);  no usages
    int getPointsCount();  no usages
    FunctionPoint getPoint(int index);  no usages
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;  no usages
    double getPointX(int index);  no usages
    void setPointX(int index, double x) throws InappropriateFunctionPointException;  no usages
    double getPointY(int index);  no usages
    void setPointY(int index, double y);  no usages
    void deletePoint(int index);  no usages
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;  no usages
}
```

Задание 7

В класс Main я добавила проверку для случаев, в которых объект табулированной функции должен выбрасывать исключения.

```
import functions.*;

public class Main {
    public static void main(String[] args) {
        double left = -2;
        double right = 3;
        int count = 6;
        TabulatedFunction arrayFunc = new ArrayTabulatedFunction(left, right, count);
        System.out.println("ArrayTabulatedFunction");
        System.out.println("Функция f(x) = 2x² + 3x - 1");
        System.out.println("Левая граница функции: " + arrayFunc.getLeftDomainBorder());
        System.out.println("Правая граница функции: " + arrayFunc.getRightDomainBorder());
        for (int i = 0; i < arrayFunc.getPointsCount(); i++) {
            double x = arrayFunc.getPointX(i);
            arrayFunc.setPointY(i, y: 2*x*x + 3*x - 1);
        }
        System.out.println("\nТочки ArrayTabulatedFunction:");
        for (int i = 0; i < arrayFunc.getPointsCount(); i++) {
            System.out.println("(" + arrayFunc.getPointX(i) + "; " + arrayFunc.getPointY(i) + ")");
        }
        System.out.println("\nПроверка значений ArrayTabulatedFunction:");
        for (double x = -3; x <= 4; x += 0.5) {
            System.out.println("f(" + x + ") = " + arrayFunc.getFunctionValue(x));
        }
        TabulatedFunction linkedFunc = new LinkedListTabulatedFunction(left, right, count);
        System.out.println("\nLinkedListTabulatedFunction");
        System.out.println("Функция f(x) = 2x² + 3x - 1");
        System.out.println("Левая граница функции: " + linkedFunc.getLeftDomainBorder());
        System.out.println("Правая граница функции: " + linkedFunc.getRightDomainBorder());
        for (int i = 0; i < linkedFunc.getPointsCount(); i++) {
            double x = linkedFunc.getPointX(i);
            linkedFunc.setPointY(i, y: 2*x*x + 3*x - 1);
        }
        System.out.println("\nТочки LinkedListTabulatedFunction:");
        for (int i = 0; i < linkedFunc.getPointsCount(); i++) {
            System.out.println("(" + linkedFunc.getPointX(i) + "; " + linkedFunc.getPointY(i) + ")");
        }
        System.out.println("\nПроверка значений LinkedListTabulatedFunction:");
        for (double x = -3; x <= 4; x += 0.5) {
            System.out.println("f(" + x + ") = " + linkedFunc.getFunctionValue(x));
        }
        double[] values = {1, -2, -1, 4, 13, 26};
        TabulatedFunction func1 = new ArrayTabulatedFunction( leftX: -2, rightX: 3, values);
        System.out.println("\nArrayTabulatedFunction с массивом значений");
        System.out.println("Функция создана с массивом значений:");
        for (int i = 0; i < func1.getPointsCount(); i++) {
            System.out.println("x=" + func1.getPointX(i) + ", y=" + func1.getPointY(i));
        }
        TabulatedFunction func2 = new LinkedListTabulatedFunction( leftX: -2, rightX: 3, values);
        System.out.println("\nLinkedListTabulatedFunction с массивом значений");
        System.out.println("Функция создана с массивом значений:");
        for (int i = 0; i < func2.getPointsCount(); i++) {
            System.out.println("x=" + func2.getPointX(i) + ", y=" + func2.getPointY(i));
        }
        System.out.println("\nОперации с ArrayTabulatedFunction");
        System.out.println("Изменяем значение у третьей точки:");
```

```
        try {
            arrayFunc.setPoint( index: 2, new FunctionPoint( x: 0,  y: -1));
            System.out.println("Новая точка 2: x = " + arrayFunc.getPointX( index: 2) + ", y = " + arrayFunc.getPointY( index: 2));
        } catch (InappropriateFunctionPointException e) {
            System.err.println("Ошибка при изменении точки: " + e.getMessage());
        }
        System.out.println("\nДобавляем новую точку (x=1.5, y=8):");
        try {
            arrayFunc.addPoint(new FunctionPoint( x: 1.5,  y: 8));
            System.out.println("Точки ArrayTabulatedFunction после добавления:");
            for (int i = 0; i < arrayFunc.getPointsCount(); i++) {
                System.out.println("(" + arrayFunc.getPointX(i) + "; " + arrayFunc.getPointY(i) + ")");
            }
        } catch (InappropriateFunctionPointException e) {
            System.err.println("Ошибка при добавлении точки: " + e.getMessage());
        }
        System.out.println("\nОперации с LinkedListTabulatedFunction");
        System.out.println("Изменяем значение у третьей точки:");
        try {
            linkedFunc.setPoint( index: 2, new FunctionPoint( x: 0,  y: -1));
            System.out.println("Новая точка 2: x = " + linkedFunc.getPointX( index: 2) + ", y = " + linkedFunc.getPointY( index: 2));
        } catch (InappropriateFunctionPointException e) {
            System.err.println("Ошибка при изменении точки: " + e.getMessage());
        }
        System.out.println("\nДобавляем новую точку (x=1.5, y=8):");
        try {
            linkedFunc.addPoint(new FunctionPoint( x: 1.5,  y: 8));
            System.out.println("Точки LinkedListTabulatedFunction после добавления:");
            for (int i = 0; i < linkedFunc.getPointsCount(); i++) {
                System.out.println("(" + linkedFunc.getPointX(i) + "; " + linkedFunc.getPointY(i) + ")");
            }
        } catch (InappropriateFunctionPointException e) {
            System.err.println("Ошибка при добавлении точки: " + e.getMessage());
        }
        System.out.println("\nНаходим значение y для 2.5 с помощью линейной интерполяции:");
        double valueAt25Array = arrayFunc.getFunctionValue( x: 2.5);
        double valueAt25Linked = linkedFunc.getFunctionValue( x: 2.5);
        System.out.println("ArrayTabulatedFunction f(2.5) = " + valueAt25Array);
        System.out.println("LinkedListTabulatedFunction f(2.5) = " + valueAt25Linked);
        System.out.println("\nУдаляем четвертую точку:");
        try {
            arrayFunc.deletePoint( index: 3);
            System.out.println("Точки ArrayTabulatedFunction после удаления:");
            for (int i = 0; i < arrayFunc.getPointsCount(); i++) {
                System.out.println("(" + arrayFunc.getPointX(i) + "; " + arrayFunc.getPointY(i) + ")");
            }
        } catch (Exception e) {
            System.err.println("Ошибка при удалении точки: " + e.getMessage());
        }
        try {
            linkedFunc.deletePoint( index: 3);
            System.out.println("Точки LinkedListTabulatedFunction после удаления:");
            for (int i = 0; i < linkedFunc.getPointsCount(); i++) {
                System.out.println("(" + linkedFunc.getPointX(i) + "; " + linkedFunc.getPointY(i) + ")");
            }
        } catch (Exception e) {
            System.err.println("Ошибка при удалении точки: " + e.getMessage());
        }
        testExceptionsWithInterface();
    }
```

```
    private static void testExceptionsWithInterface() {  1 usage
        System.out.println("Проверка исключений");
        System.out.println("ArrayTabulatedFunction:");
        TabulatedFunction arrayFunc = new ArrayTabulatedFunction( leftX: 0,  rightX: 5,  pointsCount: 3);
        testSimpleExceptions(arrayFunc);
        System.out.println("LinkedListTabulatedFunction:");
        TabulatedFunction linkedFunc = new LinkedListTabulatedFunction( leftX: 0,  rightX: 5,  pointsCount: 3);
        testSimpleExceptions(linkedFunc);
        System.out.println("Проверка исключений в конструкторах:");
        testConstructorExceptions();
    }
@   private static void testSimpleExceptions(TabulatedFunction func) {  2 usages
        try {
            func.getPoint( index: 10);
            System.out.println("Ошибка: не выброшено FunctionPointIndexOutOfBoundsException");
        } catch (FunctionPointIndexOutOfBoundsException e) {
            System.out.println("FunctionPointIndexOutOfBoundsException: " + e.getMessage());
        }
        try {
            func.addPoint(new FunctionPoint( x: 2.0,  y: 5.0));
            func.addPoint(new FunctionPoint( x: 2.0,  y: 10.0)); // Дубликат X
            System.out.println("Ошибка: не выброшено InappropriateFunctionPointException");
        } catch (InappropriateFunctionPointException e) {
            System.out.println("InappropriateFunctionPointException: " + e.getMessage());
        }
        try {
            TabulatedFunction tempFunc;
            if (func instanceof ArrayTabulatedFunction) {
                tempFunc = new ArrayTabulatedFunction( leftX: 0,  rightX: 2,  pointsCount: 3);
            } else {
                tempFunc = new LinkedListTabulatedFunction( leftX: 0,  rightX: 2,  pointsCount: 3);
            }
            tempFunc.deletePoint( index: 0);
            tempFunc.deletePoint( index: 0); // Должно остаться 1 точка - ошибка
            System.out.println("Ошибка: не выброшено IllegalStateException");
        } catch (IllegalStateException e) {
            System.out.println("IllegalStateException: " + e.getMessage());
        }
        System.out.println();
    }
    private static void testConstructorExceptions() {  1 usage
        try {
            new ArrayTabulatedFunction( leftX: 5,  rightX: 3,  pointsCount: 4); // Левая > правой
            System.out.println("Ошибка: не выброшено IllegalArgumentException");
        } catch (IllegalArgumentException e) {
            System.out.println("IllegalArgumentException (левая > правой): " + e.getMessage());
        }
        try {
            new LinkedListTabulatedFunction( leftX: 0,  rightX: 5,  pointsCount: 1); // Точек < 2
            System.out.println("Ошибка: не выброшено IllegalArgumentException");
        } catch (IllegalArgumentException e) {
            System.out.println("IllegalArgumentException (точек < 2): " + e.getMessage());
        }
    }
}
```

Проверка

```
ArrayTabulatedFunction
Функция f(x) = 2x² + 3x - 1
Левая граница функции: -2.0
Правая граница функции: 3.0

Точки ArrayTabulatedFunction:
(-2.0; 1.0)
(-1.0; -2.0)
(0.0; -1.0)
(1.0; 4.0)
(2.0; 13.0)
(3.0; 26.0)

Проверка значений ArrayTabulatedFunction:
f(-3.0) = NaN
f(-2.5) = NaN
f(-2.0) = 1.0
f(-1.5) = -0.5
f(-1.0) = -2.0
f(-0.5) = -1.5
f(0.0) = -1.0
f(0.5) = 1.5
f(1.0) = 4.0
f(1.5) = 8.5
f(2.0) = 13.0
f(2.5) = 19.5
f(3.0) = 26.0
f(3.5) = NaN
f(4.0) = NaN
```

```
LinkedListTabulatedFunction
Функция f(x) = 2x² + 3x - 1
Левая граница функции: -2.0
Правая граница функции: 3.0

Точки LinkedListTabulatedFunction:
(-2.0; 1.0)
(-1.0; -2.0)
(0.0; -1.0)
(1.0; 4.0)
(2.0; 13.0)
(3.0; 26.0)

Проверка значений LinkedListTabulatedFunction:
f(-3.0) = NaN
f(-2.5) = NaN
f(-2.0) = 1.0
f(-1.5) = -0.5
f(-1.0) = -2.0
f(-0.5) = -1.5
f(0.0) = -1.0
f(0.5) = 1.5
f(1.0) = 4.0
f(1.5) = 8.5
f(2.0) = 13.0
f(2.5) = 19.5
f(3.0) = 26.0
f(3.5) = NaN
f(4.0) = NaN
```

```
ArrayTabulatedFunction с массивом значений
Функция создана с массивом значений:
x=-2.0, y=1.0
x=-1.0, y=-2.0
x=0.0, y=-1.0
x=1.0, y=4.0
x=2.0, y=13.0
x=3.0, y=26.0
```

```
LinkedListTabulatedFunction с массивом значений
Функция создана с массивом значений:
x=-2.0, y=1.0
x=-1.0, y=-2.0
x=0.0, y=-1.0
x=1.0, y=4.0
x=2.0, y=13.0
x=3.0, y=26.0
```

```
Операции с ArrayTabulatedFunction
Изменяем значение y третьей точки:
Новая точка 2: x = 0.0, y = -1.0

Добавляем новую точку (x=1.5, y=8):
Точки ArrayTabulatedFunction после добавления:
(-2.0; 1.0)
(-1.0; -2.0)
(0.0; -1.0)
(1.0; 4.0)
(1.5; 8.0)
(2.0; 13.0)
(3.0; 26.0)
```

```
Операции с LinkedListTabulatedFunction
Изменяем значение y третьей точки:
Новая точка 2: x = 0.0, y = -1.0

Добавляем новую точку (x=1.5, y=8):
Точки LinkedListTabulatedFunction после добавления:
(-2.0; 1.0)
(-1.0; -2.0)
(0.0; -1.0)
(1.0; 4.0)
(1.5; 8.0)
(2.0; 13.0)
(3.0; 26.0)
```

```
Находим значение y для 2.5 с помощью линейной интерполяции:
ArrayTabulatedFunction f(2.5) = 19.5
LinkedListTabulatedFunction f(2.5) = 19.5

Удаляем четвертую точку:
Точки ArrayTabulatedFunction после удаления:
(-2.0; 1.0)
(-1.0; -2.0)
(0.0; -1.0)
(1.5; 8.0)
(2.0; 13.0)
(3.0; 26.0)
Точки LinkedListTabulatedFunction после удаления:
(-2.0; 1.0)
(-1.0; -2.0)
(0.0; -1.0)
(1.5; 8.0)
(2.0; 13.0)
(3.0; 26.0)
```

```
Проверка исключений
ArrayTabulatedFunction:
FunctionPointIndexOutOfBoundsException: Индекс точки выходит за границы: 10
InappropriateFunctionPointException: Точка с координатой X=2.0 уже существует
IllegalStateException: Невозможно удалить точку: должно остаться минимум 2 точки

LinkedListTabulatedFunction:
FunctionPointIndexOutOfBoundsException: Индекс за пределами списка: 10
InappropriateFunctionPointException: Точка с координатой X=2.0 уже существует
IllegalStateException: Невозможно удалить точку: должно остаться минимум 2 точки

Проверка исключений в конструкторах:
IllegalArgumentException (левая > правой): Левая граница области определения >= правой
IllegalArgumentException (точек < 2): Количество точек меньше двух
```